

Predictive KV Cache Amortization for Edge Language Model Inference on Mobile Devices

Arjit Tripathi

arjit.tripathi2023@vitstudent.ac.in

Medha Sriram

medha.sriram2023@vitstudent.ac.in

Heer Shah

shah.heer2023@vitstudent.ac.in

Dr. Shalini L

l.shalini@vit.ac.in

Abstract—On-device language models often need situational context prepended to the user’s input: location, nearby people, time of day. The prefill step that reads this context is expensive. On a OnePlus 11R running Qwen2.5-0.5B-Instruct (Q4_K_M), prefilling a 439-token context prompt took 14.2 s. Server systems cache KV state across users to skip repeated prefill, but that depends on overlapping requests from multiple users, which a single phone does not have. A phone does have sensors. CAP-KVC uses GPS, BLE proximity, and time of day to rank a small set of pre-written contexts, prefills the likely ones during idle time, and keeps up to three as resident llama.cpp KV state. At interaction time only the user’s intent tokens are decoded. With a cached 44-token context, the CAP-KVC prefill step (9 intent tokens) averaged 351 ms versus 2507 ms for the matched 52-token inline condition, a $7.1\times$ reduction ($n = 30$ each); the KV state restore added 2.4 ms. Over 100 synthetic days with five contexts, correct-context selection reached 98.5% of 6,400 interactions versus 92.2% for request-driven LRU caching, at a cost of 1.5% wrong-context selections. Undetected GPS forgery raised wrong-context selections to 38.9%. One device, one model, five contexts, synthetic traces.

Index Terms—Key-value cache, edge inference, predictive caching, language models, mobile systems, sensor-driven context.

I. INTRODUCTION

In order for a language model to respond effectively to a question or task posed by a user from their smartphone, it needs to know where the user is physically located as well as other factors about the user’s environment—for instance, who else may be in the user’s vicinity and what time of day it currently is. Take an AAC (augmentative and alternative communication) app [1]: the user taps “I need water,” and the app has to expand that into a full sentence. But what it should say depends on the setting. A hospital ward at 11 a.m. calls for different phrasing than home in the evening with a caregiver present, so the context has to be there. The standard approach is to prepend a context description to the intent and let the model read both.

However, feeding this information into the language model prior to submitting a query is very costly. We found that on an Android-based smartphone, populating a 439-token input prompt took 14.2 s (Section V-B). Even a short 52-token prompt took 2.5 s, which works out to roughly 30 ms per token. For someone who communicates through the device, that kind of delay comes up every single turn.

Server-based systems sidestep these repeated prefills using a shared key-value (KV) store to maintain state across multiple user requests. SGLang keeps prompt prefixes in a radix tree [6], vLLM shares KV blocks between requests [7], Prompt Cache stores attention states for reusable prompt modules [8], and RAGCache caches the KV state of retrieved documents [9]. These systems rely on request overlap from multiple users: one user’s prefill populates the cache, and a later user’s matching request skips it. A single phone has no such overlap. The request that would populate the cache is the user’s own first turn in a new setting, and that turn pays the full prefill cost.

While there is no stream of requests coming from a phone, the device does have sensors available to help inform which context the user is likely going to want. In this paper we propose CAP-KVC (Context-Aware Predictive KV-Cache priming). It makes predictions on the best few context descriptions based upon the user’s current location (via GPS), who or what else is nearby (via Bluetooth Low Energy, or BLE, proximity), and what time of day it is. While the phone is in idle mode it prefills those predicted contexts the user is most likely to want next. Once prefilled, the phone stores up to three of them as llama.cpp KV state. When the user decides to interact with the device, only their intent tokens need to be decoded. On a OnePlus 11R running Qwen2.5-0.5B-Instruct (Q4_K_M), we measured that restoring an existing 44-token context as opposed to prefilling it—parsing all of its tokens inline—takes 351 ms vs. 2507 ms respectively ($7.1\times$, $n = 30$ each). Restoring the actual state takes 2.4 ms.

Our study was performed over a synthetic period of 100 days of sensor data for five different contexts. The routing and residency policies were able to determine the correct context for 98.5% of 6,400 interactions generated during that time frame, compared with 92.2% for a request-driven cache with no ability to leverage sensing capabilities, which misses on every first visit. The evaluation is limited to a single device, a single model, five contexts, and synthetic traces.

This paper makes three contributions. First, a sensor-driven predictive KV caching design for a single user on a phone: deterministic context scoring from GPS, BLE and time-of-day readings, a bounded resident set with hysteresis-gated replacement, and a two-tier fallback for cold misses. Second, an Android implementation built on llama.cpp through JNI,

TABLE I
WHERE CAP-KVC DIFFERS FROM REQUEST-DRIVEN KV REUSE.

	Request-driven reuse	CAP-KVC
Setting	Multi-user server	One user, one phone
Cache filled	When a request arrives	Before it, while idle
Selection signal	Prompt prefix / document	GPS, BLE, time
First request in a context	Full prefill	Restore if predicted
Failure mode	Cold miss	Wrong context or cold miss

with per-slot KV residency, rollback of each interaction so a resident state can be reused unchanged, crash-safe cache persistence via atomic file renames, and location input validation. Third, an evaluation that separates the restore benefit (matched-context latency) from prediction quality (trace-driven comparison against oracle, request-driven, random and single-sensor baselines) and from adversarial sensor input (forged GPS and cloned BLE beacons).

II. RELATED WORK

Reusing KV state across requests. SGLang [6] stores prompt prefixes in a radix tree; vLLM [7] shares KV blocks via paged memory; Prompt Cache [8] precomputes attention states for reusable prompt modules; CacheBlend [10] fuses cached non-prefix chunks; CacheGen [11] compresses KV for cross-server transfer; RAGCache [9] caches per-document KV with retrieval-aware eviction. All populate the cache when a request containing the relevant prefix arrives, and the payoff requires later requests with overlapping content. RAGCache is closest to CAP-KVC in that it caches document-specific KV state, but it remains reactive: the first request bearing a document pays full prefill. On a single phone, there is no request stream from other users. CAP-KVC populates the cache *before* any request, using sensor evidence rather than request content (Table I). Section V-C simulates a request-driven LRU policy on the same traces. The KV save/load mechanism itself is provided by llama.cpp [2]; our contribution is choosing *which* context to prepare, *when*, and from *what* signals.

Shrinking the KV cache. H2O [12] and Scissorhands [13] evict less important tokens; SnapKV [14] compresses keys and values; StreamingLLM [15] keeps only attention sinks and recent tokens; KVQuant [16] quantizes the stored state. These reduce memory within a sequence. They are orthogonal to what we do—a compressed KV state can still be primed and restored.

On-device inference. Quantization [4], [5], small-model design [17], runtimes like llama.cpp [2] and MLC-LLM [18], flash offloading [19], and heterogeneous NPU/CPU execution [20] have collectively made it feasible to run language models on a phone. They lower the cost of processing each token, but they do not get rid of the context prefill that happens before the first output token. That is the gap CAP-KVC is meant to address.

III. SYSTEM DESIGN

The architecture has two independent paths (Fig. 1). A background sweep runs every 60 s, reads sensors, scores each context, and updates which KV states are resident. The interaction path is triggered by user input: it picks the best resident state and decodes the intent.

A. Problem

Let $C = \{c_1, \dots, c_M\}$ be pre-written contexts with prompts p_i of N_i tokens. In the straightforward approach (the retrieval-augmented style), the device decodes the full p_i followed by the user’s intent q on every request; prefill cost grows with $N_i + |q|$. CAP-KVC moves the prefill of p_i to idle time and stores the resulting KV state. At interaction time the device restores that state and decodes only q .

The question is which contexts to prepare, since memory allows only $K < M$ of them to be resident at once. Our prototype has $M = 5$ contexts (home morning, home evening, hospital ward, therapy room, caregiver visit) and $K = 3$ resident slots.

B. Context Scoring

Each context c_i is associated with an expected hour of day t_i , an anchor GPS location ℓ_i , and a set of BLE beacon anchors A_i with weights w_{ij} . When a sensor reading comes in, the system computes a score for each context:

$$S(c_i) = \alpha S_{\text{time}} + \beta S_{\text{gps}} + \gamma S_{\text{ble}}, \quad \alpha + \beta + \gamma = 1, \quad (1)$$

where each component captures one kind of sensor evidence:

$$S_{\text{time}} = \exp(-d_{\text{circ}}(t, t_i)^2 / 2\sigma^2), \quad \sigma = 2 \text{ h}, \quad (2)$$

$$S_{\text{gps}} = \exp(-d_{\text{hav}}(\ell, \ell_i) / \lambda), \quad \lambda = 0.1 \text{ km}, \quad (3)$$

$$S_{\text{ble}} = \sum_{j \in A_i} w_{ij} \mathbb{1}[j \text{ seen}] / \sum_{j \in A_i} w_{ij}. \quad (4)$$

d_{circ} wraps around midnight so that 11:55 p.m. and 12:05 a.m. are treated as ten minutes apart rather than nearly 24 hours. S_{gps} uses the Haversine distance and falls to 0.37 at 100 m. The BLE score is the weighted fraction of expected beacons that the device actually detected.

The weights α, β, γ are not learned—they come from a per-sensor reliability heuristic. Time gets a fixed $w_t = 0.4$. GPS gets $w_g = \exp(-a/50 \text{ m})$ for reported accuracy a (0 without a fix). BLE gets w_b equal to the mean of $1/(1+e^{-0.1(r+70)})$ over the observed RSSI values r (0 when no beacons are detected). These are normalized to sum to one. Contexts scoring below 0.01 are dropped. “Predictive” in the title refers to preparing a context before the request, not to a trained model.

C. Residency and Hysteresis

At most $K = 3$ KV states can be resident. With five contexts, three slots cover the majority of transitions while bounding background priming work. Section V-F evaluates $K \in \{1, 2, 3\}$.

After each sweep the scorer ranks all contexts. Candidates already resident stay. A new candidate takes a free slot if one exists; if all three are occupied, it replaces the weakest resident

Background: sweep every 60 s; BLE-free check every 15 min

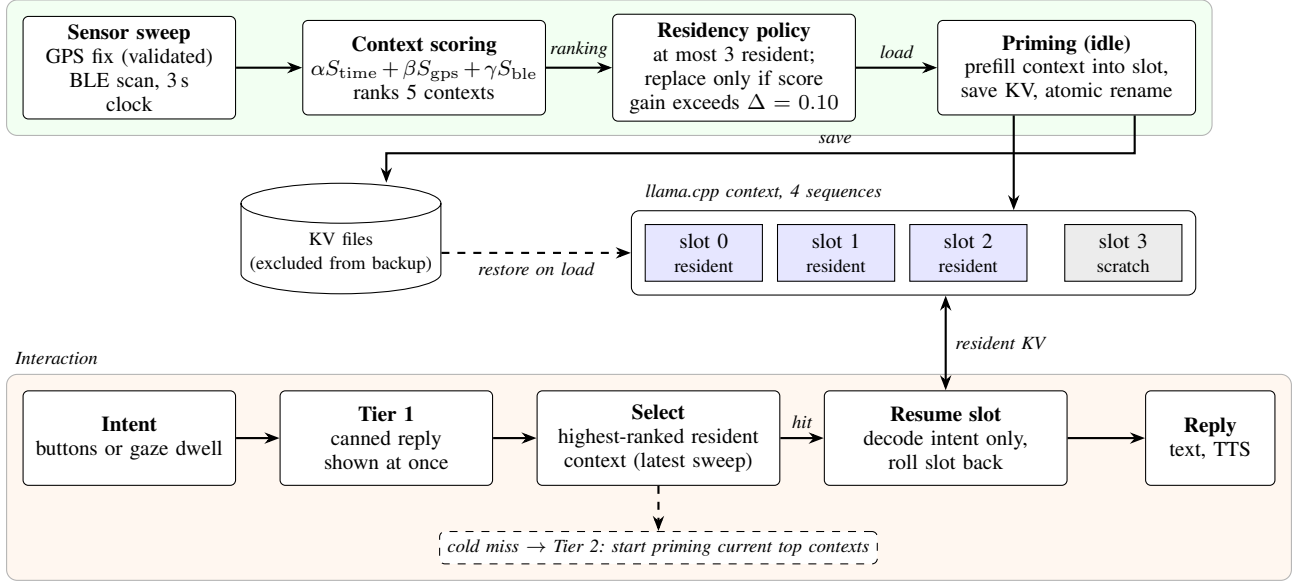


Fig. 1. CAP-KVC architecture. A background sweep scores five pre-written contexts from GPS, BLE and time, keeps up to three of them as primed KV state in llama.cpp sequence slots, and saves each state to disk. An interaction shows a canned reply, then resumes the highest-ranked resident context and decodes only the intent; if none is resident, priming starts in the background. Slot 3 serves inline inference without disturbing resident states.

only when its score exceeds the weakest’s by more than Δ . Without a margin, noisy GPS accuracy and intermittent BLE detection caused repeated state swaps between sweeps. K denotes the number of resident KV states. Δ is a replacement margin on scores and is independent of K . The deployed value is $\Delta = 0.10$; Section V-D measures its effect.

The same replacement rule runs in the 60-second sweep and in a 15-minute background check that omits BLE scanning and may only add the top-ranked context.

D. Interaction and Fallback

When the user interacts, the app picks the highest-ranked resident context from the latest sweep. If one exists, the device resumes that state and decodes only the intent. If no ranked context is resident (a cold miss), a canned response goes out immediately (Tier 1) and priming of the top-ranked contexts starts in the background (Tier 2). A wrong-context answer is the other failure mode: a resident context is used but does not match the user’s actual situation.

IV. IMPLEMENTATION

AACBridge is an Android app (Kotlin, API 26+, arm64-v8a) that uses llama.cpp [2] through JNI. The model is Qwen2.5-0.5B-Instruct [3] quantized to Q4_K_M; we chose it as the smallest instruction-tuned model available in llama.cpp’s GGUF format that fits within the phone’s memory constraints.

Native boundary. 14 JNI methods, all scalar arguments and results: lifecycle (initialize backend, load model, release), KV operations (save, load, clear scratch, reset slot), inference (inline run, prefill only, resume), and four timing getters (prefill ms, generation ms, prompt tokens, generated

tokens). A single `ReentrantLock` serializes every native call; llama.cpp context state is shared and not thread-safe.

Sequence slots. We set up the llama.cpp context with four sequences of 1024 tokens each. Three of these are reserved for resident states, and the fourth serves as a scratch space for inline inference and benchmark runs. We added the scratch sequence specifically so that running a benchmark or doing inline inference would not disturb a resident state that was already loaded.

Priming decodes the context into its assigned slot and serializes the slot to a temporary file, then renames atomically (no partial cache file on crash). Resuming appends the intent at the slot’s current position, generates the response, then strips everything past the cached boundary—rolling the slot back so the resident state is reused unchanged next time.

With 24 layers, 2 KV heads of dimension 64, and fp16 storage, Qwen2.5-0.5B needs 12 KiB of KV data per token. The four sequences reserve 48 MiB at context creation. We did not re-measure process memory after this layout change.

Sensing. A foreground service runs the sensor sweep every 60 s. Each sweep performs a 3-second BLE scan and requests a location fix. Every location provider’s last fix is checked before the system trusts it: the fix must not be flagged as mock by the Android OS, it must be at most 120 s old, and the reported accuracy must fall between 2 and 500 m. If nothing passes, one fresh fix is requested (network provider preferred over GPS for speed). A WorkManager job runs every 15 minutes without BLE.

Intent input. On-screen buttons or gaze-dwell interface. An sEMG+gaze fusion classifier from earlier work is included as a prototype input but not evaluated here.

TABLE II

NATIVELY TIMED LATENCY COMPONENTS (MS), MEDIAN [IQR], $n = 30$ EACH. CAP-KVC RESTORES A 44-TOKEN CONTEXT AND DECODES THE 9 INTENT TOKENS.

Condition (tokens)	Prefill	Generation	Restore
No context (9)	377 [288, 503]	5775 [2823, 6213]	—
Inline, matched (52)	2449 [2231, 2777]	7964 [7350, 9374]	—
Inline (89)	4116 [3677, 4487]	8589 [7822, 11294]	—
Inline (206)	7075 [5148, 8700]	6991 [4107, 10275]	—
Inline (439)	14024 [13219, 14941]	5912 [5467, 7003]	—
CAP-KVC (9)	331 [276, 355]	4395 [3985, 5110]	1.2 [1.0, 3.3]

Backup. App backup disabled. KV cache directory excluded from cloud backup and device-to-device transfer.

V. EVALUATION

A. Setup

All latency measurements were taken on a OnePlus 11R (Snapdragon 8+ Gen 1, 8 GB RAM, Android 14), with 4 threads, batch size 512, greedy decoding, and a cap of 64 output tokens. Each condition ran 2 warm-up trials followed by 30 measured trials. Prefill and generation times come from `std::chrono` timestamps placed around the `llama_decode` call in native code; restore time is measured around the load call. All 180 trials and the analysis script are in the repository.

During this run, the gaze interface remained active and issued 45 interaction inferences between trials. These held the engine lock, inflating the wall-clock totals of some trials with lock-wait time unrelated to inference. We report only the natively timed components (prefill, generation, restore), which the lock wait does not affect, and omit end-to-end totals. The benchmark code now blocks interaction inference during measurement and acquires the lock before starting the trial timer.

B. Matched-Context Latency

This experiment measures the prefill cost removed when a previously prepared context is restored rather than decoded from scratch.

The CAP-KVC condition holds a 44-token context in its cache. The matched inline condition decodes the same context plus the intent as a single 52-token prompt. Larger inline conditions (89, 206, and 439 tokens) show how inline prefill scales; their ratios relative to CAP-KVC grow by construction since the inline prompts are longer, and they are not matched comparisons.

Table II shows the results. The KV restore overhead (loading the cached state from disk) averaged 2.4 ms (median 1.2 ms). The CAP-KVC prefill (decoding only the 9 intent tokens after restore) averaged 351 ms (SD 97). The matched inline prefill (all 52 tokens) averaged 2507 ms (SD 522). The ratio is $7.1\times$ (Welch $t(31.0) = 22.2$, $p < 10^{-19}$; Mann-Whitney $p < 10^{-10}$).

A rough time to first token (restore + prefill + one generation step) is about 0.43 s for CAP-KVC, about 2.6 s inline.

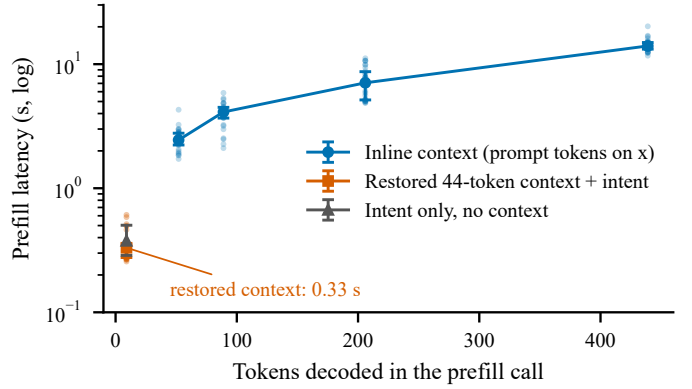


Fig. 2. Prefill latency against tokens decoded (log scale). Points are trials; markers show median and IQR ($n = 30$ per condition).

Inline prefill grew at about 30 ms per prompt token (OLS, $R^2 = 0.91$; Fig. 2), reaching 14.2 s at 439 tokens where the CAP-KVC prefill was $40.5\times$ shorter. That larger ratio reflects the cost CAP-KVC avoids when contexts happen to be long, not a matched speedup. We only measured restore for the 44-token cache and cannot say how it scales with context length.

The prefill saving should be kept in perspective. Generation dominated the full response: 4.4–8.6 s at median with 64 output tokens. So the 2.1 s saved in matched prefill shortens the overall response by much less than $7.1\times$; the benefit is concentrated in time to first token. Generation times varied across conditions and trials (IQRs above 1 s). We recorded no thermal or CPU-frequency data and cannot explain this variation.

C. Prediction and Cache Policy

Traces. We do not have recorded sensor traces from real users, so we generated synthetic ones (seed 42). Each of 100 simulated days follows the five-context routine 06:00–22:00, with each context boundary shifted per day by $U(\pm 0.5 \text{ h})$. A sweep runs every simulated minute; an interaction fires every 15 minutes (6,400 interactions total). Per-sweep noise parameters: GPS displacement $\mathcal{N}(0, 50 \text{ m})$, GPS dropout 10%, reported accuracy $\mathcal{N}(15, 10) \text{ m}$, per-beacon detection 80% with RSSI $\mathcal{N}(-65, 10) \text{ dBm}$, BLE outage 15%, spurious beacon 10%. The high-noise variant: $\pm 1.5 \text{ h}$ boundary shifts, 200 m GPS displacement, 30% GPS dropout, 50 m accuracy, 50% detection, 40% BLE outage. The context time anchors were written from the same daily routine that the traces follow, so time-of-day evidence is favourable by construction. Section V-H evaluates schedules with shifted and jittered boundaries using independent seeds.

Simulation. The trace evaluator runs the production scoring, residency and selection code (the same code deployed on the phone). It does not model priming delay: a state chosen at one sweep is assumed usable by the next simulated minute.

Metrics. For each interaction, let c^* be the true context and $\hat{c}$ be the one the system actually used. Correct: $\hat{c} = c^*$. Wrong context: $\hat{c} \neq c^*$ (the system picked a resident context but the wrong one). Cold miss: no ranked context is resident. These

TABLE III
INTERACTION OUTCOMES OVER 6,400 SYNTHETIC INTERACTIONS (%)
AND STATES LOADED PER DAY. $\Delta = 0.10$ UNLESS NOTED.

Policy / scenario	Correct	Wrong	Cold	Loads/d
Oracle (always resident)	100.0	0.0	0.0	5.0
Request-driven LRU, no sensing	92.2	0.0	7.8	5.0
Random ranking	20.7	79.3	0.0	1011
Time only	92.2	7.8	0.0	5.0
GPS only	87.9	12.1	0.0	45.7
BLE only	92.6	6.9	0.5	115.8
All sensors (deployed)	98.5	1.5	0.0	31.6
All sensors, $\Delta = 0$	98.5	1.5	0.0	65.9
GPS 70% unavailable	98.2	1.8	0.0	116.7
High noise	90.6	9.4	0.0	106.2
GPS forged, not detected	61.1	38.9	0.0	18.2
GPS forged, mock flag rejected	98.3	1.7	0.0	75.8
Cloned BLE beacons	90.1	9.9	0.0	103.2

partition every interaction into exactly one outcome. Loads per day counts how many times a state was primed or restored into the resident set—a measure of background work.

Baselines. *Oracle:* c^* always resident (upper bound). *Request-driven LRU:* no sensing; the correct context becomes known only at request time, is used if cached, otherwise is a cold miss and enters a three-entry LRU cache persisting across days. *Random:* uniform random scores per sweep, same residency rule. *Single sensor:* time only; GPS only or BLE only with $w_t = 0.001$ so time does not leak through normalization.

With all sensors, the deployed rules picked the correct context 98.5% of the time (Table III).

Request-driven LRU scored 92.2% correct and never selected a wrong context—by definition, since it only uses a context it has verified. Its 7.8% failure rate is entirely cold misses. Five contexts visited in order with three cache entries means every first visit after the cache fills is a miss. Sensing eliminated those first-visit misses, but introduced 1.5% wrong-context selections.

GPS alone could not separate the three contexts at the home location. BLE alone left the system with no candidate in 0.5% of interactions (no registered beacon detected). Time alone reached 92.2%, but that number reflects how the traces were built—the time anchors match the daily schedule—not a property we would expect from real user schedules.

Cold misses were near zero for every sensing policy. The router’s top-1 accuracy matched the share of correct interactions in every configuration. With three of five contexts always resident, the dominant failure mode is a wrong selection rather than a miss, so Hit@3-style metrics carry little information here. Random ranking also had no cold misses (three of five are always resident) but was correct only 20.7% of the time.

D. Hysteresis

Fig. 3 shows what happened when we varied Δ with everything else held fixed. Raising Δ from 0 to the deployed 0.10 halved loads per day under normal noise (65.9 to 31.6) and cut them from 297.5 to 106.2 under high noise. Accuracy

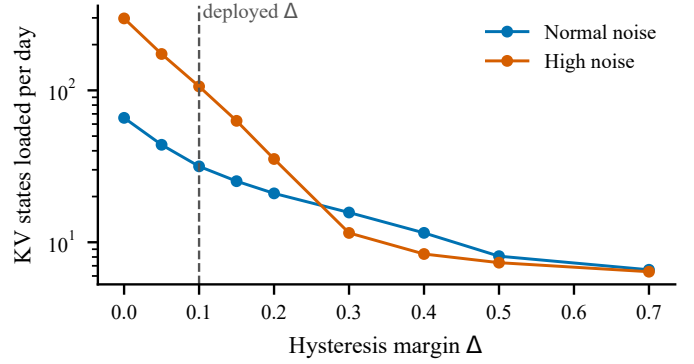


Fig. 3. States loaded per day against hysteresis margin Δ (100 synthetic days each). The share of correct interactions stayed at 98.5% (normal) and 90.6% (high noise) across all margins up to 0.5.

did not change at any margin up to 0.5—consistent with the interpretation that genuine context changes move the scores by much more than these margins. Each load is a disk restore or (if the cache file is missing) a full prefill, so the margin mainly saves background work. A margin of 0.3 would reduce loads further on these traces at no accuracy cost, but we keep 0.10 until real traces are available; a larger margin also delays switching when sensor evidence shifts by a small but genuine amount.

E. Markov Successor Baseline

A deterministic five-context routine visited in the same order every day is trivially predictable: the successor of `home_morning` is always `hospital_ward`. We trained a first-order Markov chain $P(\text{next} | \text{current})$ on the first 50 days and tested on the remaining 50. The Markov predictor achieved 99.7% correct context selection with only 6.0 loads/day, compared with 98.5% correct and 31.6 loads/day for the full CAP-KVC router. This result is expected: on these synthetic traces, context ordering is perfectly deterministic, and the Markov baseline exploits this structure without any sensor input. On real schedules with skipped, reordered, or interrupted contexts, the advantage would diminish. We report this baseline to make the ceiling explicit.

F. Residency Capacity (K) Ablation

With $K = 1$, the system carries a single resident state and changes it at every transition: 99.4% correct, 0.6% wrong context, 10.7 loads/day. With $K = 2$: 98.5% correct, 1.5% wrong, 39.1 loads/day. With $K = 3$ (deployed): 98.5% correct, 1.5% wrong, 31.6 loads/day. $K = 1$ achieves the highest accuracy because every slot change is a deliberate transition to the current best state, with no stale residents competing. However, every transition requires a load (restore or prime), and each load costs seconds of background work. $K = 3$ amortizes loads across transitions: the new context is often already resident from a previous sweep. On these traces, accuracy is equivalent for $K \geq 2$.

G. Multi-Seed Evaluation

To test robustness to noise realization, we ran the full router on 20 independent seeds (each 100 days, normal noise). Correct-context rate ranged from 98.0% (seed 8) to 98.6% (seed 19), mean 98.3%, SD 0.15 pp. LRU was constant at 92.2% (deterministic given the schedule). The advantage of sensing over LRU (+6.1 pp mean) was stable across all seeds.

H. Independent Trace Schedules

We tested robustness to schedule variation using traces with ± 1 h boundary shifts (5 seeds), ± 2 h shifts, doubled interaction frequency (7-minute intervals), and combined high jitter (± 1.5 h, 150 m GPS). Mean correct rates: 97.7% (± 1 h), 95.6% (± 2 h), 98.3% (variable dwell), 95.9% (high jitter). Performance degrades predictably with boundary ambiguity: at ± 2 h shifts, context boundaries overlap more with adjacent time anchors, and the wrong-context rate rises to 4.3% (vs. 1.5% at ± 0.5 h).

I. Abstention and Risk-Coverage

If the score gap between the top-ranked and second-ranked context is small, the system could abstain (fall back to Tier-1 generic response). We sweep a margin threshold τ from 0 to 0.20. At $\tau = 0$ (no abstention): 98.5% correct, 1.5% wrong. At $\tau = 0.05$: 97.1% correct, 0.8% wrong, 2.1% abstention. At $\tau = 0.10$: 94.6% correct, 0.4% wrong, 4.9% abstention. The abstention mechanism trades correct selections for a reduction in wrong-context responses. Whether this trade-off is worthwhile depends on the cost of wrong context versus no context, which is application-specific. CAP-KVC does not currently use abstention; these results show it is available if needed.

J. Not Measured

Battery drain, power, temperature, CPU frequency: unmeasured for the 60-second sweep, the 15-minute job, and background priming. Each load in Table III is a unit of background work whose energy cost we have not been able to quantify.

VI. THREAT MODEL

Two assets matter: the context selection (it determines what the device says for the user) and the cached KV files (they encode the full context prompts).

We consider three attackers. (A) An app on the same phone supplying mock locations via Android’s API. (B) Someone nearby broadcasting BLE beacons that clone registered anchor identifiers. (C) Someone with file-level access to the phone or backups.

Scope of attack. The router only ranks the five pre-written contexts. Sensor manipulation can force a wrong selection but cannot add a context, change a prompt, or alter the user’s intent. A wrong context still changes what the model produces, so the damage is bounded but not eliminated.

Mitigations and residual risk. We reject location fixes flagged as mock by Android, along with stale fixes and those

with reported accuracy outside 2–500 m. With the mock flag working, forged GPS had almost no effect (1.7% wrong context, Table III). On a rooted phone a forged fix need not carry the flag; if it also reports plausible accuracy (e.g. 5 m), our checks accept it, and the wrong-context rate reached 38.9%. BLE advertisements are unauthenticated—anyone can broadcast any beacon identifier—and we have no defence against cloned beacons (9.9% wrong context). The router does not cross-check GPS against BLE. Cache files are stored unencrypted in app-private storage, excluded from backup and device transfer; protection extends only as far as the Android sandbox.

Out of scope. Rooted/compromised OS, kernel or firmware attackers, physical extraction of storage.

VII. LIMITATIONS AND CONCLUSION

The latency measurements come from one phone (One-Plus 11R), one 0.5B-parameter model, 30 trials per condition, one cached context size. Restore time was measured only for a 44-token cache; we do not know how it scales with context length. The benchmark ran on a debug build with the gaze interface active (Section V); a controlled rerun with randomized trial order and disabled interaction inference is needed. We recorded no thermal or CPU-frequency data during trials. Memory footprint with the model loaded and three resident KV states: 627 MB proportional (PSS), of which 151 MB native heap (llama.cpp context and KV tensors) and 391 MB memory-mapped model file. On the 8 GB OnePlus 11R this leaves headroom but constrains multi-tasking; smaller models or KV compression would reduce it. The prediction results come from synthetic traces whose time anchors match the contexts’ schedule; on these traces, a Markov successor baseline that uses no sensors at all achieves 99.7% correct (Section V-E), exposing that the deterministic schedule is trivially predictable. The simulation does not model priming delay—a context chosen at one sweep is assumed available at the next. Real multi-day sensor recordings from actual users are needed. Five hand-written contexts. Energy cost unmeasured. No evaluation with AAC users. No clinical claim.

Within those limits, the measurements do separate the parts of the question. The KV restore overhead was 2.4 ms; the matched prefill dropped from 2507 ms to 351 ms. Sensor-driven residency selected the correct context for 98.5% of synthetic interactions (mean 98.3% across 20 seeds, SD 0.15 pp; Section V-G), compared with 92.2% for request-driven LRU, by trading first-visit cold misses for 1.5% wrong-context selections. With ± 2 h schedule jitter, accuracy degraded to 95.6% (Section V-H). Abstention at $\tau = 0.05$ reduced wrong-context responses from 1.5% to 0.8% at the cost of 2.1% cold misses (Section V-I). The main vulnerability is undetected GPS forgery, which raised wrong-context answers to 38.9%.

Code, benchmark logs, evaluation harness, and figure scripts are available at <https://github.com/omni-ar/AACBridge>.

ACKNOWLEDGMENT

The authors used Claude (Anthropic) to draft the text of this revised manuscript (all sections), which the authors then edited; to help write and revise code in the Android application, the JNI layer, the trace evaluator and the analysis and plotting scripts used for Tables II–III and Figs. 2–3; and for language editing. Google Antigravity was used as an AI-assisted development tool for parts of the Android/JNI implementation (Section IV). Fig. 1 is drawn with TikZ code, written with Claude’s assistance from a component layout specified by the authors; Figs. 2–3 are rendered from measured or simulated data by scripts in the repository. No figure was produced by an image-generation model. Research questions, system design decisions, experiment choices and the interpretation of results are the authors’.

REFERENCES

- [1] D. R. Beukelman and J. C. Light, *Augmentative and Alternative Communication: Supporting Children and Adults with Complex Communication Needs*, 5th ed. Baltimore, MD, USA: Paul H. Brookes, 2020.
- [2] G. Gerganov *et al.*, “llama.cpp: LLM inference in C/C++,” 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [3] A. Yang *et al.*, “Qwen2.5 technical report,” *arXiv:2412.15115*, 2024.
- [4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Proc. NeurIPS*, 2022.
- [5] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proc. ICLR*, 2023.
- [6] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Proc. NeurIPS*, 2024.
- [7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [8] I. Gim, G. Chen, S.-S. Lee, N. Sarda, A. Khandelwal, and L. Zhong, “Prompt Cache: Modular attention reuse for low-latency inference,” in *Proc. MLSys*, 2024.
- [9] C. Jin, Z. Zhang, X. Jiang, F. Liu, S. Liu, X. Liu, and X. Jin, “RAG-Cache: Efficient knowledge caching for retrieval-augmented generation,” *ACM Trans. Comput. Syst.*, vol. 44, no. 1, Art. 2, Feb. 2026, doi: 10.1145/3768628.
- [10] J. Yao *et al.*, “CacheBlend: Fast large language model serving for RAG with cached knowledge fusion,” in *Proc. EuroSys*, 2025.
- [11] Y. Liu *et al.*, “CacheGen: KV cache compression and streaming for fast large language model serving,” in *Proc. ACM SIGCOMM*, 2024.
- [12] Z. Zhang *et al.*, “H₂O: Heavy-hitter oracle for efficient generative inference of large language models,” in *Proc. NeurIPS*, 2023.
- [13] Z. Liu *et al.*, “Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time,” in *Proc. NeurIPS*, 2023.
- [14] Y. Li *et al.*, “SnapKV: LLM knows what you are looking for before generation,” in *Proc. NeurIPS*, 2024.
- [15] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *Proc. ICLR*, 2024.
- [16] C. Hooper *et al.*, “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in *Proc. NeurIPS*, 2024.
- [17] Z. Liu *et al.*, “MobileLLM: Optimizing sub-billion parameter language models for on-device use cases,” in *Proc. ICML*, 2024.
- [18] MLC Team, “MLC LLM,” 2023. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [19] K. Alizadeh *et al.*, “LLM in a flash: Efficient large language model inference with limited memory,” in *Proc. ACL*, 2024.
- [20] Z. Xue *et al.*, “PowerInfer-2: Fast large language model inference on a smartphone,” *arXiv:2406.06282*, 2024.